ShieldDOCS - Format-Preserving PII Redaction Engine

Complete Masterclass Report: Assignment Problem, Technical Architecture & Line-by-Line Code Explanation

Author: Engineering & AI Systems Group

Target Repository: Singhkunall/pii-doc-redactor

Date: August 2026

Live Production App: pii-doc-redactor.onrender.com

Part 1: Assignment Statement & Core Engineering Problem

The objective of this assignment was to build an enterprise-grade document processing system capable of automatically detecting and redacting Personally Identifiable Information (PII) from legal, corporate, and financial Word documents (e.g., Red Herring Prospectuses, Corporate NDAs, HR agreements) and text files, substituting sensitive data with realistic, consistent fake replacements.

The Fundamental Challenge with Naive Redaction:

In Microsoft Word XML (`.docx`), character formatting (bold, italic, font family, font size, text color, highlight) is stored inside child node elements called **Runs (`<w:r>`)**. Standard python-docx scripts perform text replacement by overwriting paragraph.text = new_text. This naive approach **erases all child Run XML elements**, stripping bold headers, italic legal notes, and table layouts into plain unstyled text. ShieldDOCS solves this by introducing **run-aware XML character mapping**.

Part 2: Technical Architecture & Design Principles

PII Engine	Python 3.11, Regex, Custom Legal Heuristics	High precision detection without heavy NLP model overhead
Format Preserver	OpenXML Run-Aware Character Offset Mapper	Retains 100% of bold/italic/colors & table structures
Fake Generator	Stateless MD5 Hash Seeding (`FakeMapper`)	Guarantees 100% entity consistency across 50+ page documents
Backend API	FastAPI, CORSMiddleware, 1-Shot Response	Single HTTP round-trip for instant <1s UI rendering
Web Frontend	Vanilla JS, Glassmorphic CSS, Blob Download	Interactive live preview, category filters, 1-click download
Cloud Host	Gunicorn + Multi-Worker Uvicorn on Docker/Render	Production concurrency and dynamic PORT env binding

Part 3: Exhaustive Code Walkthrough & Explanation

3.1 `pii_engine.py` - Core Detection & Run-Level Manipulation

A. Structured Regex Detectors:

• EMAIL_RE: Uses word boundaries \b to match emails without capturing surrounding text.

• PHONE_RE: Dual-pattern regex matching Indian +91 numbers and US (555) 019-2834 formats.

• DOB_RE: Employs contextual triggers (Date of Birth: or DOB:) before date parsing to avoid redacting standard contract execution dates.

B. Unstructured Legal Heuristics & Stopword Filtering:

• COMPANY_SUFFIX_RE: Matches capital-cased entity names preceding legal suffixes (Private Limited, LLP, Inc).

• NAME_TRIGGER_RE: Extracts names following executive titles (*Managing Director*, *Compliance Officer*, *Authorised Signatory*).

• LEGAL_DEFINED_TERM_WORDS & _is_probable_person_name(): Filters candidates against a 150+ legal prospectus stopword dictionary to prevent terms like *Book Running Lead Managers* from being misidentified as human names.

C. Deterministic Hash Generator (`FakeMapper`):

• Uses MD5 hash seeding: seed = int(md5(salt + category + value).hexdigest(), 16).

• **Why:** Guarantees that 'Rahul Kapoor' is 100% consistently replaced by 'Kavya Reddy' across a 50-page document without requiring a database.

D. Format-Preserving Run Redactor (`redact_paragraph_preserve_runs`):

1. Constructs a character-to-run offset map: run_map = [(run_obj, char_offset_in_run)].
2. Sorts PII spans from **right to left** (reverse=True) so replacements do not alter character indices of earlier spans.
3. Modifies text inside individual run.text nodes, preserving parent <w:rPr> character styles (bold, italic, colors).

3.2 `main.py` - FastAPI Server & 1-Shot Performance Optimization

• **CORSMiddleware**: Exposes Content-Disposition headers so web browsers can extract stream filenames.

• **1-Shot Response (`/api/analyze`)**: Combines text extraction, PII scanning, entity table generation, AND HTML preview creation into 1 single HTTP request. Reduces UI load latency from 4s down to <1s.

• **Dynamic Port Binding**: Reads os.environ.get('PORT', 10000) dynamically to prevent 502 Bad Gateway errors on Render/Docker.

3.3 `static/app.js` - Robust Frontend UI Logic

• **JavaScript String `.endsWith()` Fix**: Replaced Python .endswith with JS .endsWith() to prevent frontend script crashes.

• **Safe Error Parsing**: Wraps res.json() in fallback try-catch blocks to handle non-JSON 502/500 proxy responses safely without throwing Unexpected end of JSON input.

• **Blob URL Download**: Converts binary stream into window.URL.createObjectURL(blob) and triggers programmatic browser download.

Part 4: Technical Summary & Comparison Matrix

Challenge	Implemented Solution	Why We Chose It	Alternative & Disadvantage
Style Loss	Run-Level Offset Mapper	Modifies text inside `` while preserving `` formatting	Overwriting `paragraph.text` erases all bold/italic/color XML nodes
Entity Consistency	MD5 Hash Seeding (`FakeMapper`)	Rahul Kapoor maps to Kavya Reddy 100% consistently everywhere	Random generators replace same person with different names across pages
False Positives	Legal Stopword Filter Set	Prevents prospectus boilerplate from being flagged as human names	Heavy NLP models (spaCy) take 500MB+ RAM and slow down boot times
UI Latency	Unified 1-Shot Endpoint	Returns metrics, table, and preview HTML in 1 single HTTP call	2 sequential network calls introduced 4s delay on free cloud servers
502 Bad Gateway	Dynamic Port + Gunicorn	Binds `0.0.0.0:$PORT` and spawns 2 async Uvicorn worker threads	Hardcoded port 8000 causes proxy gateway routing failures on Render

Part 5: 2-Minute Technical Pitch / Summary

"I built ShieldDOCS, an enterprise format-preserving PII redaction engine for Word documents and legal text files. The primary challenge was that standard redactors wipe out Word XML Run elements, destroying bold headers, italic notes, font colors, and table structures. I solved this by developing a Run-level XML manipulator in Python that maps string offsets to individual Word Run nodes, replacing sensitive text while preserving 100% of character styles.

For detection, I built a hybrid engine combining regexes for structured PII with contextual heuristics and legal dictionary filters for unstructured PII. For replacement, I designed a stateless MD5 hash-seeded generator that guarantees entity consistency across multi-page documents without needing a database. Finally, I wrapped this in an optimized 1-shot FastAPI backend paired with a Glassmorphic Web UI and deployed it using Gunicorn with Uvicorn workers on Docker and Render."